Exercises - Binary representations

Adolfo Villafiorita

September 23, 2025

Contents

1 Intro **1**

2 Sizeof **1**
2.1 Solution: Sizes 2
2.2 Solution: Lowest/Highest 3
2.3 Use the documentation, Luke 3

3 Bitwise logic operators **3**

4 Conversion **4**

5 Hexadecimal **4**

6 ... and up to base 36 **5**

7 More Exercises (at Home?) **6**

1 Intro

These are some exercises meant to explore and understand the way in which variables are stored by a computer.

2 Sizeof

You can get the size of a variable using `sizeof`.

1. Write a program which outputs the size of all the variable types you are aware of (beware of signed and unsigned)

2. Now take one variable, set it to the maximum value it can store and add

   (a) What happens? (check with signed and unsigned)

3. Using what you know from theory, check the minimum and maximu values you can set for a var

2.1 Solution: Sizes

We print them all, but we are interested in `short` and `unsigned short`.

```
#include <iostream>
using namespace std;

int main() {
  cout << sizeof(unsigned int) << endl;
  cout << sizeof(int) << endl;
  cout << sizeof(unsigned short) << endl;
  cout << sizeof(short) << endl;
  cout << sizeof(unsigned long) << endl;
  cout << sizeof(long) << endl;
}
```

The binary is the most direct representation of variables in the system. We write a program which initializes variables using the binary format and, then, we inspect it.

Steps:

1. We find the size used by CPP to store short (both signed and unsigned) using sizeof. **This allows us to write number in binary (we know how many digits we need)**

2. we identify the lowest and highest number (in binary) we can represent using the size allocated by CPP (We use the table in the next slide to deduce the numbers)

3. we declare a var holding the lowest (highest) number, using the binary representation

4. we print them out: they will be output in decimal

2.2 Solution: Lowest/Highest

Number Stored	Complement 2 (if negative) (so that I know the module)	Intepretation in Base 10 (if signed)
000		0
001		1
010		2
011		3
100	011+1 = 100 (4)	-4
101	010+1 = 011 (3)	-3
110	001+1 = 010 (2)	-2
111	000+1 = 001 (1)	-1

2.3 Use the documentation, Luke

There is a simpler solution:

```
/* Doc here: https://cplusplus.com/reference/climits/ */
#include <climits>
```

Doc here: https://cplusplus.com/reference/climits/.

3 Bitwise logic operators

C++ has some operators to allow bit-by-bit operations on data.
Doc here: https://en.cppreference.com/w/cpp/language/operator$_{\text{arithmetic}}$

- `~` NOT
- `<<` Shift left
- `>>` Shift right
- `&` Bitwise AND (different from LOGICAL AND)
- `|` Bitwise OR (different from LOGICAL OR)
- `^` Bitwise XOR

Moreover, you can also represent numbers in various bases, including binary (0b) and hexadecimal (0x).
For instance:

- Binary: `int a = 0b0000'1111`
- Hexadecimal: `int b = 0xAA`

You can separate digits with `'` (it has no semantics, it is just a visual aid):

- `'` (simple quote) can be used to separate digits
- Using the information above, write code which computes the 2-complement of a number. Print the result to cout. If need be, set the variable using the binary format.
- Write some code which performs the bitwise operations (and, xor, «). What does « do on a number?

4 Conversion

Using a cycle, write a program which asks for a number, a base and prints the digits required to represent the number in the given base.

Example. Given 16 and 2 as input, the program will output:

```
16, 2
16 | 0
   | 0
   | 0
   | 0
   | 1
```

Now try with a base greater than 10. What are the "symbols" you need to represent the numbers?

5 Hexadecimal

A base commonly used in Computer Science is the hexadecimal base (base 16).

In order to write numbers using a single symbol per digit, the following convention is used:

- `A` is used when the digit is `10`
- `B` is used when the digit is `11`

- `C` is used when the digit is `12`
- `D` is used when the digit is `13`
- `E` is used when the digit is `14`
- `F` is used when the digit is `15`

Thus `AB1`, for instance, represents the number `10*16^2 + 11*16^1 + 1*16^0`.

To distinguish hexadecimal numbers, numbers are prefixed by `0x`

The base is very convenient, since each digit represents 4 bits and you can use to hexadecimal digits to represent a byte:

`0xC8 => 1100 0100`

Modify the program above so that it prints the correct hexadecimal letters to cout.

6 ... and up to base 36

Why stopping at base 16? Try with bases higher than 16 (up to 36) and use the letters of the alphabet to encode the values higher than 16.

Thus, for instance, in base 20:

- `A` is used when the digit is `10`
- `B` is used when the digit is `11`
- `C` is used when the digit is `12`
- `D` is used when the digit is `13`
- `E` is used when the digit is `14`
- `F` is used when the digit is `15`
- `G` is used when the digit is `16`
- `H` is used when the digit is `17`
- `I` is used when the digit is `18`
- `J` is used when the digit is `19`

Write a program which represents up to base 36. Check the values in input. Output the correct letters.

(Remarks: these bases are not used in practice, but they make a nice exercise.)

7 More Exercises (at Home?)

- How do you multiply by 4 using the shift operators?
- How do you divide by 2 using the shift operators?
- Given two binary numbers you want to add, how to you know the bits which will require carry over? (e.g. the positions at which both numbers have a 1)